

```
import { Request, Response, NextFunction } from 'express'
import jwt from 'jsonwebtoken'
import prisma from '../lib/prisma'
import { getRedisClient } from '../lib/redis'
import { JWT_SECRET } from '../lib/config'
import { AuthErrorCodes, AuthErrorCode, ErrorResponse } from '../lib/errorCodes'

interface SafeUser {
  id: string
  phone: string | null
  email: string | null
  username: string | null
  nickname: string | null
  avatar: string | null
  userTier: string
}

export type AuthRequest = Request & { user?: SafeUser }

export type AuthenticatedRequest = Request & { user: SafeUser }

const USER_CACHE_TTL_SECONDS = 300

function userCacheKey(userId: string): string {
  return `lc:user:${userId}:safe`
}

async function getCachedUser(userId: string): Promise<SafeUser | null> {
  const redis = getRedisClient()
  if (!redis) return null
  try {
    const raw = await redis.get(userCacheKey(userId))
    if (!raw) return null
    return JSON.parse(raw) as SafeUser
  } catch {
    return null
  }
}

async function setCachedUser(user: SafeUser): Promise<void> {
  const redis = getRedisClient()
  if (!redis) return
  try {
    await redis.setex(userCacheKey(user.id), USER_CACHE_TTL_SECONDS, JSON.stringify(user))
  }
```

```
    } catch {
      // 缓存写入失败不影响主流程
    }
  }

export const authenticate = async (req: AuthRequest, res: Response, next: NextFunction) => {
  try {
    const token = req.headers.authorization?.replace('Bearer ', '')

    if (!token) {
      return errorResponse(res, 401, AuthErrorCodes.UNAUTHORIZED)
    }

    const decoded = jwt.verify(token, JWT_SECRET) as { userId: string }

    // 优先从 Redis 缓存读取用户信息
    let safeUser = await getCachedUser(decoded.userId)

    if (!safeUser) {
      const user = await prisma.user.findUnique({
        where: { id: decoded.userId }
      })

      if (!user) {
        return errorResponse(res, 401, AuthErrorCodes.USER_NOT_FOUND)
      }

      // 检查用户状态
      if (user.status === 'banned') {
        return errorResponse(res, 403, AuthErrorCodes.ACCOUNT_BANNED)
      }
      if (user.status === 'suspended') {
        return errorResponse(res, 403, AuthErrorCodes.ACCOUNT_SUSPENDED)
      }
      // 检查账号是否被锁定
      if (user.lockedUntil && new Date() < user.lockedUntil) {
        return errorResponse(res, 403, AuthErrorCodes.ACCOUNT_LOCKED)
      }
    }

    safeUser = {
      id: user.id,
      phone: user.phone,
      email: user.email,
      username: user.username,
    }
  }
}
```

```
        nickname: user.nickname,
        avatar: user.avatar,
        userTier: user.userTier,
    }

    // 写入 Redis 缓存（异步，不阻塞请求）
    setCachedUser(safeUser).catch(() => {})
}

req.user = safeUser
next()
} catch (error) {
    return errorResponse(res, 401, AuthErrorCodes.TOKEN_INVALID)
}
}

/**
 * 分享文本智能解析服务
 * 从手机 APP 分享文案中提取 URL、还原短链、提取标题、识别平台
 */
import fetch from 'node-fetch'
import https from 'https'
import * as cheerio from 'cheerio'
import { detectPlatform } from './platforms'

// HTTPS Agent: 允许自签名证书（部分海外站点证书不受信任）
const httpsAgent = new https.Agent({
    rejectUnauthorized: false,
})

// ===== 类型定义 =====

export interface ParseResult {
    /** 提取/还原后的最终 URL */
    url: string
    /** 原始提取的 URL（短链），如果与最终 URL 不同则有值 */
    originalUrl?: string
    /** 从分享文本中提取的标题 */
    textTitle?: string
    /** 识别的平台 */
    platform: string
    /** 解析步骤详情（用于调试） */
    steps: ParseStep[]
}
```

```
export interface ParseStep {
  step: string
  status: 'success' | 'failed' | 'skipped'
  detail?: string
  result?: string
}

// ===== 短链域名配置 (V3.2 扩充) =====

const SHORT_LINK_DOMAINS = [
  // 国内视频/直播
  'v.douyin.com',      // 抖音
  'live.douyin.com',   // 抖音直播
  'b23.tv',            // B 站 (新增)
  'm.bilibili.com',    // B 站移动端
  'v.kuaishou.com',    // 快手
  'v.qq.com',          // 腾讯视频短链
  'm.tb.cn',           // 淘宝
  '3.cn',              // 京东短链 (新增)
  't.cn',              // 微博
  'weibo.cn',          // 微博移动端

  'dwz.cn',            // 百度
  'url.cn',            // 腾讯
  'u.jd.com',          // 京东
  'p.pinduoduo.com',   // 拼多多
  'mobile.yangkeduo.com', // 拼多多移动端
  'oem.toutiao.com',   // 今日头条
  // 国内工具/社交
  'weixin://',         // 微信 (URL Scheme)
  'xhslink.com',        // 小红书短链
  'music.163.com',      // 网易云音乐
  'xima.xiami.com',     // 虾米音乐 (已停服但有存量)
  // 海外视频/社交
  'vm.tiktok.com',      // TikTok
  'vt.tiktok.com',      // TikTok
  'tiktok.com',         // TikTok 主站短链
  'youtu.be',          // YouTube 短链
  'fb.com',             // Facebook
  'l.facebook.com',     // Facebook 重定向
  'lnkd.in',            // LinkedIn 短链
  'pin.it',             // Pinterest 短链
  'redd.it',            // Reddit 短链
]
```

```
'instagram.com',    // Instagram
'instagr.am',      // Instagram 短链
't.me',            // Telegram
'bit.ly',          // 通用短链
'tinyurl.com',     // 通用短链
't.co',            // Twitter/X
'goo.gl',          // Google 短链（已停服但仍有存量）
'amzn.to',         // Amazon 短链
'a.co',            // Amazon 短链
'ow.ly',           // Hootsuite 短链
'buff.ly',         // Buffer 短链
'lnk.to',          // 通用短链
'soo.link',        // 通用短链
'sourl.cn',        // 缩链
'mulu.cn',         // 木鲁短链
'shorturl.at',     // 通用短链
]

// 跟踪参数（URL 标准化时移除，V3.2 扩充）
const TRACKING_PARAMS = [
  // 通用 Analytics
  'utm_source', 'utm_medium', 'utm_campaign', 'utm_term', 'utm_content',
  'utm_source', 'utm_source', // 重复确保匹配
  // 通用追踪
  'from', 'sharefrom', 'share_token', 'share_id', 'source', 'is_from_webapp',
  'mid', 'aid', 'author_id', 'scene', 'type', 'channel',
  'sp_atk', 'sp_bid', 'sp_cid',
  'spm', 'scm',    // 淘宝/天猫追踪

  'ref', 'referrer', 'refer', 'referer',
  'ns', 'mrd',    // 小红书追踪
  'enterfrom', 'enterFrom', // 抖音
  'enterMethod',
  'groupId', 'itemId', // 快手
  'f', 's',        // 通用追踪
  'app', 'platform', 'os',
  'sign', 'ts', 'timestamp',
  'tracelog', 'tracklog',
  'cstm', 'custom',
  // V3.2 新增追踪参数
  'alginfo',    // 优酷/土豆追踪
  'seqId',      // 优酷序列追踪
  'abId',       // A/B 测试
  'sceneId',    // 场景追踪
```

```

'reqlId',      // 请求追踪
's',          // 短哈希追踪
'vfm',        // 视频格式追踪
'fv',         // 视频追踪
'wid',        // 头条追踪
'page', 'pageId', 'page_id',
'share', 'shareId', 'share_id',
'account', 'accountId',
'user', 'userId', 'user_id', 'uid',
'trace', 'tracelog', 'track_id',
'campaign', 'campaign_id',
'ad', 'adId', 'ad_id', 'ads',
'click', 'clickId', 'click_id',
'keyword', 'keywords', 'search_key',
'q', 'query', 'search_query',
// UUID 格式追踪参数
'uuid', 'sid', 'session_id', 'sessionid',
'cid', 'client_id', 'clientId',
// 时间戳追踪参数
'_t', 't', 'time', 'timeStamp', 'timestamp_ms',
'nocache', 'rand', 'r',
]

// ===== 平台模板正则（用于清理分享文案中的模板文字，V3.2 大幅扩充）
=====

const PLATFORM_TEMPLATE_PATTERNS = [
  // ===== 抖音 =====
  /\[d.\]+\s*复制打开抖音[, ]?\s*/g,
  /复制打开抖音[, ]?\s*/g,
  /复知打开抖音[, ]?\s*/g,
  /打开抖音看看[, ]?\s*/g,
  /抖音[: :]\s*/g,

  /打开抖音\s*直接观看/g,
  /查看置顶评论[\s\S]*?抖音/g,
  /在抖音记录美好生活/g,

  // ===== 小红书 =====
  /复制打开小红书[, ]?\s*/g,
  /打开小红书 App[, ]?\s*/g,
  /打开小红书[, ]?\s*/g,
  /小红书[: :]\s*/g,
  /小红书 App[, ]?\s*/g,

```



```
/B 站[: :]?s*/g,  
/【bilibili】/g,  
  
//===== 优酷/腾讯/爱奇艺 =====  
/复制打开优酷[, ]?s*/g,  
/打开优酷[, ]?s*/g,  
/优酷[: :]?s*/g,  
/复制打开腾讯视频[, ]?s*/g,  
/打开腾讯视频[, ]?s*/g,  
/腾讯视频[: :]?s*/g,  
/复制打开爱奇艺[, ]?s*/g,  
/打开爱奇艺[, ]?s*/g,  
/爱奇艺[: :]?s*/g,  
  
//===== 微信/公众号 =====  
/复制打开微信[, ]?s*/g,  
/打开微信[, ]?s*/g,  
/微信[: :]?s*/g,  
/公众号[: :]?s*/g,  
/微信公众平台/g,  
  
//===== 知乎 =====  
/复制打开知乎[, ]?s*/g,  
/打开知乎[, ]?s*/g,  
/知乎[: :]?s*/g,  
/知乎，圆环的答案/g,  
  
//===== 微博 =====  
/复制打开微博[, ]?s*/g,  
/打开微博[, ]?s*/g,  
/微博[: :]?s*/g,  
/分享微博[, ]?s*/g,  
/微博，随时随地发现新鲜事/g,  
  
//===== 今日头条 =====  
/复制打开头条[, ]?s*/g,  
/打开头条[, ]?s*/g,  
/今日头条[: :]?s*/g,  
/头条[: :]?s*/g,  
  
//===== 网易云音乐 =====  
/复制打开网易云音乐[, ]?s*/g,  
/打开网易云音乐[, ]?s*/g,  
/网易云音乐[: :]?s*/g,
```



```

/云音乐[: :]?s*/g,

// ===== 通用模板（放在最后作为兜底） =====
/复制打开[s\S]*?[ ,]/g,
/【[s\S]*?】/g,
/点击链接[s\S]*?[ ,]/g,

/长按复制[s\S]*?[ ,]/g,
/分享[^n]*?[到给][^n]*?[ ,]/g,
/打开 APP[ ,]?s*/g,
/查看更多[ ,]?s*/g,
/了解更多[ ,]?s*/g,
/直接点击链接[ ,]?s*/g,
/点击链接打开[ ,]?s*/g,
/链接:[^n]+/g,
/视频地址:[^n]+/g,
/播放地址:[^n]+/g,
]

// ===== URL 提取 =====

// 主 URL 正则：匹配 http/https 开头的链接
const URL_REGEX =
/https?:\/\/[^\s\u4e00-\u9fff\u3000-\u303f\u0000-\uffef\u08\u09\u0a\u0b\u0c\u0e\u0f1f]+|www\.[^\s\u4e00-\u9fff\u3000-\u303f\u0000-\uffef\u08\u09\u0a\u0b\u0c\u0e\u0f1f]+/gi

// 备用域名正则：不带 http 前缀的常见域名
const DOMAIN_REGEX =
/[a-z0-9-]+\.(com|cn|net|org|io|cc|tv|fm|me|app|dev|top|vip|shop)[\/\?][^\s\u4e00-\u9fff\u3000-\u303f\u0000-\uffef]*/gi

/**
 * 从混合文本中提取 URL
 */
export function extractUrlFromText(text: string): { url: string; steps: ParseStep[] } {
  const steps: ParseStep[] = []

  // 1. 尝试主正则提取
  let urls: string[] = text.match(URL_REGEX) || []

  if (urls.length > 0) {
    steps.push({ step: '提取 URL', status: 'success', detail: '从文本中提取到标准 URL', result: urls[0] })
  }

```

```
    } else {
      // 2. 尝试备用域名正则
      const domainMatches = text.match(DOMAIN_REGEX) || []
      urls = domainMatches.map((d: string) => d.startsWith('http') ? d : `https://${d}`)

      if (urls.length > 0) {
        steps.push({ step: '提取 URL', status: 'success', detail: '从文本中提取到裸域名', result:
urls[0] })
      } else {
        steps.push({ step: '提取 URL', status: 'failed', detail: '未在文本中找到有效链接' })
        return { url: '', steps }
      }
    }
  }

  // 3. 清理尾部特殊字符
  const firstUrl = urls[0] || ''
  let cleanUrl = firstUrl.replace(/[^\]]*>.,;:!?""', °;: ! ? \ ] ] ) 》 " " … — ~ ~ ] + $ / , " )

  return { url: cleanUrl, steps }
}

// ===== 短链还原 =====

/**
 * 判断 URL 是否为短链
 */
export function isShortLink(url: string): boolean {
  try {
    const hostname = new URL(url).hostname
    return SHORT_LINK_DOMAINS.some(d => hostname === d || hostname.endsWith('.' + d))
  } catch {
    return false
  }
}

/**
 * 还原短链为真实 URL
 * 策略: 先 HEAD 请求跟随重定向, 失败则 GET 请求跟随重定向, 再失败则解析 HTML 中
  的 location.href
 */
export async function resolveShortLink(shortUrl: string): Promise<{ resolvedUrl: string; steps:
ParseStep[] }> {
  const steps: ParseStep[] = []
```

```
if (!isShortLink(shortUrl)) {
  steps.push({ step: '短链还原', status: 'skipped', detail: '非短链域名，无需还原' })
  return { resolvedUrl: shortUrl, steps }
}

const UA = 'Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 like Mac OS X) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/16.0 Mobile/15E148 Safari/604.1'

// 策略 1: HEAD 请求跟随重定向
try {
  const response = await fetch(shortUrl, {
    method: 'HEAD',
    redirect: 'follow',
    headers: { 'User-Agent': UA },
    signal: AbortSignal.timeout(5000),
    agent: shortUrl.startsWith('https') ? httpsAgent : undefined,
  })
  if (response.url && response.url !== shortUrl) {
    steps.push({ step: '短链还原', status: 'success', detail: 'HEAD 请求跟随重定向成功',
result: response.url })
    return { resolvedUrl: response.url, steps }
  }
} catch {
  // HEAD 失败，继续尝试 GET
}

// 策略 2: GET 请求跟随重定向
try {
  const response = await fetch(shortUrl, {
    method: 'GET',
    redirect: 'follow',
    headers: { 'User-Agent': UA },
    signal: AbortSignal.timeout(8000),
    agent: shortUrl.startsWith('https') ? httpsAgent : undefined,
  })

  if (response.url && response.url !== shortUrl) {
    steps.push({ step: '短链还原', status: 'success', detail: 'GET 请求跟随重定向成功', result:
response.url })
    return { resolvedUrl: response.url, steps }
  }
}

// 策略 3: 解析 HTML 中的 JavaScript 跳转（如淘宝 m.tb.cn）
const contentType = response.headers.get('content-type') || "
```

```
    if (contentType.includes('text/html')) {
      const html = await response.text()
      const resolvedFromHtml = extractRedirectUrl(html, shortUrl)
      if (resolvedFromHtml && resolvedFromHtml !== shortUrl) {
        steps.push({ step: '短链还原', status: 'success', detail: '从 HTML 中解析到跳转 URL',
result: resolvedFromHtml })
        return { resolvedUrl: resolvedFromHtml, steps }
      }
    }
  } catch {
    // GET 也失败
  }

  steps.push({ step: '短链还原', status: 'failed', detail: '短链还原失败，使用原始 URL' })
  return { resolvedUrl: shortUrl, steps }
}

/**
 * 从 HTML 中提取跳转 URL（V3.2 增强版）
 * 支持：meta refresh、多种 JS 跳转模式、下载链接提取、平台专属正则
 */
function extractRedirectUrl(html: string, baseUrl: string): string | null {
  const $ = cheerio.load(html)

  // 1. meta refresh: <meta http-equiv="refresh" content="0;url=xxx">
  const metaRefresh = $('meta[http-equiv="refresh"]').attr('content')
  if (metaRefresh) {
    const urlMatch = metaRefresh.match(/url=(.+)/i)
    if (urlMatch) {
      return resolveRelativeUrl(baseUrl, urlMatch[1].trim().replace(/["]/, ''))
    }
  }

  // 2. JavaScript location.href / window.location / location.replace（增强版）
  const jsPatterns = [
    // 标准跳转
    /location\.href\s*=\s*"([^\"]+)"/,
    /window\.location\s*=\s*"([^\"]+)"/,
    /location\.replace\s*\(\s*"([^\"]+)\s*"(\s*"([^\"]+)\s*"*)*\)/,
    /window\.location\.href\s*=\s*"([^\"]+)"/,

    // 增强跳转（V3.2 新增）
    /top\.location\.href\s*=\s*"([^\"]+)"/,
    /top\.location\s*=\s*"([^\"]+)"/,
  ]
```

```

/document\location\href\s*=\s*["']([^\"]+)[']/,
/document\location\s*=\s*["']([^\"]+)[']/,
/self\location\href\s*=\s*["']([^\"]+)[']/,
/parent\location\href\s*=\s*["']([^\"]+)[']/,
/window\location\assign\s*\(\s*["']([^\"]+)[']\s*\)/,
/window\location\replace\s*\(\s*["']([^\"]+)[']\s*\)/,
/location\assign\s*\(\s*["']([^\"]+)[']\s*\)/,
// 异步跳转（延迟执行）
/setTimeout\s*\(\s*["']?\s*location[^\n]+[']?\s*,\s*\d+\s*\)/gi,
/setTimeout\s*\(\s*function\s*\(\s*["']?\s*location[^\n]+[']?\s*\)/gi,
]
for (const pattern of jsPatterns) {
  const match = html.match(pattern)
  if (match && match[1]) {
    const url = match[1].trim()
    if (url && !url.startsWith('javascript:')) {
      return resolveRelativeUrl(baseUrl, url)
    }
  }
}

// 3. data 属性中的 URL（V3.2 新增）
const dataUrlSelectors = [
  '[data-uri]', '[data-href]', '[data-link]', '[data-redirect]',
  '[data-target]', '[data-src]', '[data-urls]',
]
for (const selector of dataUrlSelectors) {
  const dataUrl = $(selector).attr(selector.replace(/\[|\]/g, ''))
  if (dataUrl && dataUrl.startsWith('http')) {
    return dataUrl
  }
}

// 4. 下载/跳转按钮链接（V3.2 新增）
const downloadLinkSelectors = [
  'a.download-btn', 'a.download-link', 'a[download]',
  'a.open-app-btn', 'a.jump-btn', 'a.redirect-btn',
  '.download-btn[href]', '.jump-link[href]', '#downloadLink', '#jumpLink',
]
for (const selector of downloadLinkSelectors) {
  const link = $(selector).attr('href')
  if (link && link.startsWith('http') && !link.includes('javascript:')) {
    return resolveRelativeUrl(baseUrl, link)
  }
}

```

```
}
```

```
// 5. 快手专属跳转正则 (V3.2 新增)
```

```
const kuaishouPatterns = [  
  /"realUrl"\s*:\s*"([^"]+)"/,  
  /"uri"\s*:\s*"([^"]+)"/,  
  /"playUrl"\s*:\s*"([^"]+)"/,  
  /kwai:\V\/([^\s"<>]+)/g,  
]  
for (const pattern of kuaishouPatterns) {  
  const match = html.match(pattern)  
  if (match && match[1]) {  
    const url = match[1].trim()  
    if (url.startsWith('http')) {  
      return url  
    }  
  }  
}  
}
```

```
// 6. 抖音/头条专属跳转正则 (V3.2 新增)
```

```
const douyinPatterns = [  
  /"play_addr"\s*:\s*\{[^}]*"url"\s*:\s*"([^"]+)"/,  
  /"video_url"\s*:\s*"([^"]+)"/,  
  /aweme\V\/([^\s"<>]+)/gi,  
]  
for (const pattern of douyinPatterns) {  
  const match = html.match(pattern)  
  if (match && match[1]) {  
    const url = match[1].trim()  
    if (url.startsWith('http')) {  
      return url  
    }  
  }  
}  
}
```

```
// 7. 小红书专属跳转正则 (V3.2 新增)
```

```
const xiaohongshuPatterns = [  
  /"xhs_link"\s*:\s*"([^"]+)"/,  
  /"noteLink"\s*:\s*"([^"]+)"/,  
  /detail~\V\/([^\s"<>]+)/g,  
]  
for (const pattern of xiaohongshuPatterns) {  
  const match = html.match(pattern)  
  if (match && match[1]) {
```

```
        const url = match[1].trim()
        if (url.startsWith('http')) {
            return url
        }
    }
}

// 8. 从 script 标签中提取 JSON 数据中的 URL (V3.2 新增)
const scriptPatterns = [
    /window\.__INITIAL_STATE__\s*=\s*\{\^[^}]*"videoUrl"\s*:\s*"([^\"]+)"/i,
    /window\.__INITIAL_STATE__\s*=\s*\{\^[^}]*"playUrl"\s*:\s*"([^\"]+)"/i,
    /window\.__SSR_RENDER_DATA__\s*=\s*\{\^[^}]*"playUrl"\s*:\s*"([^\"]+)"/i,
]

for (const pattern of scriptPatterns) {
    const match = html.match(pattern)
    if (match && match[1]) {
        return resolveRelativeUrl(baseUrl, match[1])
    }
}

return null
}

/**
 * 将相对 URL 转为绝对 URL
 */
function resolveRelativeUrl(base: string, relative: string): string {
    try {
        return new URL(relative, base).href
    } catch {
        return relative
    }
}

// ===== 标题提取 =====

/**
 * 从分享文本中提取标题
 * 去除 URL、模板文字，提取有意义的文本作为标题
 */
export function extractTitleFromText(text: string, extractedUrl: string): { title: string; steps: ParseStep[] } {
    const steps: ParseStep[] = []
```

```
// 1. 去除 URL 部分
let title = text.replace(URL_REGEX, "").trim()

// 2. 如果提取了 URL，也尝试去除（可能 URL 格式略有差异）
if (extractedUrl) {
  title = title.replace(extractedUrl.replace(/[\.*+?^$(){}|\[\]\\\]/g, '\\$&'), "").trim()
}

// 3. 应用平台模板正则清理
for (const pattern of PLATFORM_TEMPLATE_PATTERNS) {
  title = title.replace(pattern, "").trim()
}

// 4. 清理前导数字/标点（如"2.84 "、"7.12 "等版本号格式）
title = title.replace(/^\s\d.:\/\\-]+/, "").trim()

// 5. 清理尾部残留标点
title = title.replace(/[\s, , , : , ! , ? , 、 , … , — , ~ , ~ ]+$/, "").trim()

// 6. 截断过长标题
if (title.length > 100) {
  title = title.substring(0, 100)
}

if (title) {
  steps.push({ step: '提取标题', status: 'success', detail: '从分享文本中提取标题', result:
title })
} else {
  steps.push({ step: '提取标题', status: 'failed', detail: '未能从分享文本中提取有效标题' })
}

return { title, steps }
}

// ===== 输入类型判断 =====

/**
 * 判断输入是标准 URL 还是混合文本（分享文案）
 */
export function isShareText(input: string): boolean {
  const trimmed = input.trim()

  // 纯 URL：以 http://或 https://开头且不含中文
```



```
if (/^https?:\/\/[^\s\u4e00-\u9fff]+$/.test(trimmed)) {
    return false
}

// 包含中文或空格，视为分享文本
if (/[\u4e00-\u9fff]/.test(trimmed) || /\s/.test(trimmed.trim())) {
    return true
}

// 以 www 开头且无空格，视为标准 URL
if (/^www\.[^\s]+$/.test(trimmed)) {
    return false
}

// 默认视为分享文本
return trimmed.includes(' ') || trimmed.length > 200
}

// ===== URL 标准化 =====

/**
 * 移除 URL 中的跟踪参数，补全协议（V3.2 增强版）
 * 支持：静态追踪参数、动态 UUID 参数、时间戳参数、短哈希参数
 */
function normalizeUrl(urlStr: string): string {
    try {
        // 补全协议
        let normalized = urlStr
        if (normalized.startsWith('www.')) {
            normalized = `https://${normalized}`
        }

        const urlObj = new URL(normalized)

        // 移除跟踪参数（V3.2 增强）
        const paramsToRemove: string[] = []

        const currentTime = Date.now()
        const oneYearMs = 365 * 24 * 60 * 60 * 1000

        urlObj.searchParams.forEach((value, key) => {
            const lowerKey = key.toLowerCase()

            // 1. 精确匹配的静态追踪参数
```

```
    if (TRACKING_PARAMS.includes(lowerKey)) {
        paramsToRemove.push(key)
        return
    }

    // 2. 动态参数清理 (V3.2 新增)
    // 2.1 UUID 格式 (32 位十六进制或带横线的 UUID)
    if (/^[0-9a-f]{32}$/i.test(value) ||
/^([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})$/i.test(value)) {
        paramsToRemove.push(key)
        return
    }

    // 2.2 时间戳参数 (值为 10 位或 13 位数字, 且接近当前时间或过去 1 年内)
    if (/^\d{10,13}$/i.test(value)) {
        const ts = parseInt(value, 10)
        const tsMs = value.length === 10 ? ts * 1000 : ts
        if (tsMs > 0 && tsMs < currentTime + oneYearMs) {
            paramsToRemove.push(key)
            return
        }
    }

    // 2.3 短哈希参数 (8-32 位字母数字混合, 且无明显语义)
    // V3.5 fix: 'v' is YouTube/Bilibili video ID, must be preserved
    if (lowerKey === 'v') return
    if (/^[a-z0-9]{8,32}$/i.test(value) && !isSemanticWord(value)) {
        paramsToRemove.push(key)
        return
    }

    // 2.4 base64 编码的追踪参数 (常见于 Facebook/Google Analytics)
    if (/^[A-Za-z0-9+/=]{20,}$/i.test(value)) {
        try {
            const decoded = atob(value)
            if (decoded.includes('utm_') || decoded.includes('fbclid') ||
decoded.includes('gclid')) {
                paramsToRemove.push(key)
                return
            }
        } catch {
            // 不是有效的 base64
        }
    }
}
```

```
    })

    paramsToRemove.forEach(key => urlObj.searchParams.delete(key))

    // 移除空的查询字符串
    const result = urlObj.toString()
    return result.endsWith('?') ? result.slice(0, -1) : result
  } catch {
    return urlStr
  }
}

/**
 * 判断字符串是否有语义的词（非随机哈希）
 */
function isSemanticWord(str: string): boolean {
  // 常见语义词列表
  const semanticWords = ['home', 'index', 'about', 'contact', 'product', 'detail', 'list', 'search',
    'user', 'login', 'register', 'admin', 'test', 'demo']
  const lower = str.toLowerCase()
  return semanticWords.some(word => lower.includes(word))
}

// ===== 统一解析入口 =====

/**
 * 智能解析分享文本或 URL
 * 自动判断输入类型，依次执行：提取 URL → 还原短链 → 提取标题 → 识别平台
 */
export async function parseShareInput(input: string): Promise<ParseResult> {
  const allSteps: ParseStep[] = []
  const trimmed = input.trim()

  if (!trimmed) {
    return {
      url: '',
      platform: 'other',
      steps: [{ step: '输入检查', status: 'failed', detail: '输入为空' }],
    }
  }

  // 1. 判断输入类型
  const isText = isShareText(trimmed)
  allSteps.push({
```

```
      step: '输入类型判断',
      status: 'success',
      detail: isText ? '识别为分享文本' : '识别为标准 URL',
    })

    let cleanUrl: string
    let textTitle = ''

    if (isText) {
      // 2a. 从混合文本中提取 URL
      const { url, steps: extractSteps } = extractUrlFromText(trimmed)
      allSteps.push(...extractSteps)

      if (!url) {
        return { url: '', platform: 'other', steps: allSteps }
      }
      cleanUrl = url

      // 3. 提取标题
      const { title, steps: titleSteps } = extractTitleFromText(trimmed, url)
      allSteps.push(...titleSteps)
      textTitle = title
    } else {
      // 2b. 直接作为 URL 使用
      cleanUrl = trimmed.startsWith('www.') ? `https://${trimmed}` : trimmed
      allSteps.push({ step: '提取 URL', status: 'success', detail: '直接使用输入的 URL', result:
cleanUrl })
    }

    // 4. 短链还原
    const { resolvedUrl, steps: resolveSteps } = await resolveShortLink(cleanUrl)
    allSteps.push(...resolveSteps)
    console.log('[parseShareInput] resolvedUrl=' + resolvedUrl)

    // 5. 识别平台
    const platform = detectPlatform(resolvedUrl)
    allSteps.push({ step: '识别平台', status: 'success', detail: `识别为 ${platform}`, result:
platform })

    // 6. URL 标准化（移除跟踪参数）
    const normalizedUrl = normalizeUrl(resolvedUrl)
    console.log('[normalizeUrl] in=' + resolvedUrl + ' out=' + normalizedUrl)
    if (normalizedUrl !== resolvedUrl) {
      allSteps.push({ step: 'URL 标准化', status: 'success', detail: '已移除跟踪参数', result:
```

```
normalizedUrl })
    }

    console.log('[parseShareInput] FINAL url=' + normalizedUrl)
    return {
        url: normalizedUrl,
        originalUrl: cleanUrl !== normalizedUrl ? cleanUrl : undefined,
        textTitle: textTitle || undefined,
        platform,
        steps: allSteps,
    }
}

import { Router } from 'express'
import { body, query, validationResult } from 'express-validator'
import prisma from '../lib/prisma'
import { Prisma } from '@prisma/client'
import { authenticate, AuthenticatedRequest } from '../middleware/auth'
import { DEFAULT_LIST_KEY, DEFAULT_LIST_DESC } from '../lib/config'
import { detectPlatform, getSupportedPlatformList } from '../services/platforms'
import { classifyUrl } from '../services/pageClassifier'
import { fetchUrlMetadata } from '../services/metadata'
import { parseShareInput } from '../services/share-parser'
import { checkQuota, checkQuotaBatch, invalidateQuotaCache } from '../services/quota'
import { enqueueMetadataFetch } from '../services/metadata-queue'
import { recordCollectionCreated } from '../services/prom-metrics'
import { emitEvent } from '../lib/eventBus'

import fetch from 'node-fetch'
import { CollectionErrorCodes, ListErrorCodes, CommonErrorCodes, UploadErrorCodes, QuotaErrorCodes, errorResponse } from '../lib/errorCodes'
import { sanitizeCollection, ensureHttps } from '../lib/utills'
import logger from '../lib/logger'
import { isURL } from 'validator'

// ===== HTML 导入/导出辅助函数 =====

/**
 * 解析 Netscape Bookmark Format HTML
 * 返回扁平化的收藏列表，带 listName（超过 3 层扁平化到第 3 层）
 */
function parseBookmarkHtml(html: string): { url: string; title: string; listName: string | null; coverImage?: string }[] {
    const results: { url: string; title: string; listName: string | null; coverImage?: string }[] = []
```

```
const folderStack: { name: string; depth: number }[] = []
let dlDepth = 0

for (const rawLine of html.split(/\r?\n/)) {
  const line = rawLine.trim()
  if (!line) continue

  if (/<DL>/i.test(line)) {
    dlDepth++
    continue
  }
  if (/<\DL>/i.test(line)) {
    dlDepth = Math.max(0, dlDepth - 1)
    while (folderStack.length > 0 && folderStack[folderStack.length - 1].depth > dlDepth) {
      folderStack.pop()
    }
    continue
  }

  const h3Match = line.match(/<H3[^>]*>(.*?)<\H3>/i)
  if (h3Match) {
    const name = h3Match[1].replace(/<[^>]+>/g, '').trim()
    if (!name) continue
    if (folderStack.length >= 3) {
      folderStack[2] = { name, depth: dlDepth }
    } else {
      folderStack.push({ name, depth: dlDepth })
    }
    continue
  }

  const aMatch = line.match(/<A\s+([^\s]*)>(.*?)<\A>/i)
  if (aMatch) {
    const attrs = aMatch[1]
    const title = aMatch[2].replace(/<[^>]+>/g, '').trim()

    const hrefMatch = attrs.match(/HREF=["']([^\s"]+)"'/i)
    const url = hrefMatch ? hrefMatch[1].trim() : ''

    const iconMatch = attrs.match(/ICON=["']([^\s"]+)"'/i)
    const coverImage = iconMatch ? iconMatch[1].trim() : undefined

    if (url && url.startsWith('http')) {
      const listName = folderStack.length > 0 ? folderStack[folderStack.length - 1].name : null
    }
  }
}
```

```
        results.push({ url, title, listName, coverImage })
    }
}

return results
}

/**
 * 生成 Netscape Bookmark Format HTML
 */
function generateBookmarkHtml(collections: { id: string; title: string; url: string; coverImage?:
string | null; createdAt: Date; lists: { name: string }[] }[], lists: { name: string }[], includeCover:
boolean = false): string {
    // 按列表分组
    const byList = new Map<string, typeof collections>()
    const noList: typeof collections = []

    for (const c of collections) {
        if (c.lists && c.lists.length > 0) {
            for (const l of c.lists) {
                if (!byList.has(l.name)) byList.set(l.name, [])
                byList.get(l.name)!.push(c)
            }
        } else {
            noList.push(c)
        }
    }

    const lines: string[] = [
        '<!DOCTYPE NETSCAPE-Bookmark-file-1>',
        '<META HTTP-EQUIV="Content-Type" CONTENT="text/html; charset=UTF-8">',
        '<TITLE>Bookmarks</TITLE>',
        '<H1>Bookmarks</H1>',
        '<DL><p>',
    ]

    // 无分组的收藏
    if (noList.length > 0) {
        lines.push('    <DT><H3>Ungrouped</H3>')
        lines.push('    <DL><p>')
        for (const c of noList) {
            const addDate = Math.floor(c.createdAt.getTime() / 1000)
```

```

        const coverAttr = (includeCover && c.coverImage) ? `
        ICON="${escapeHtml(c.coverImage)}" ` : "
        lines.push(`
                                <DT><A    HREF="${escapeHtml(c.url)}"
ADD_DATE="${addDate}"${coverAttr}>${escapeHtml(c.title || 'Untitled')}</A>`)
        }
        lines.push('    </DL><p>')
    }

    // 有分组的收藏
    for (const [listName, items] of byList) {
        lines.push(`    <DT><H3>${escapeHtml(listName)}</H3>`)
        lines.push('    <DL><p>')
        for (const c of items) {
            const addDate = Math.floor(c.createdAt.getTime() / 1000)
            lines.push(`
                                <DT><A    HREF="${escapeHtml(c.url)}"
ADD_DATE="${addDate}">${escapeHtml(c.title || '无标题')}</A>`)
            }
            lines.push('    </DL><p>')
        }

        lines.push('</DL><p>')
        return lines.join('\n')
    }

    function escapeHtml(str: string): string {
        return str
            .replace(/&/g, '&amp;')
            .replace(/</g, '&lt;')
            .replace(/>/g, '&gt;')
            .replace(/"/g, '&quot;')
    }

    const router = Router()

    // 注意：分组筛选只匹配直接关联的收藏，不包含子分组中的收藏
    // 每个分组只展示属于该分组自身的收藏

    /**
     * 构建收藏查询的 where 条件（复用于 offset 分页和游标分页）
     */

    async function buildCollectionWhere(
        userId: string,
        filters: {

```



```
    tagId?: string
    listId?: string
    directListId?: string
    platform?: string
    platforms?: string
    search?: string
    hasRating?: string
  }
): Promise<Prisma.CollectionWhereInput> {
  const where: Prisma.CollectionWhereInput = { userId, deletedAt: null }

  if (filters.tagId) {
    where.tags = { some: { id: filters.tagId } }
  }

  if (filters.platform) {
    where.platform = filters.platform
  }

  if (filters.platforms) {
    const platformArray = filters.platforms.split(',').filter(Boolean)
    if (platformArray.length > 0) {
      where.platform = { in: platformArray }
    }
  }

  if (filters.listId) {
    const allLists = await prisma.list.findMany({
      where: { userId },
      select: { id: true, parentId: true },
    })

    const childrenMap = new Map<string, string[]>()
    for (const list of allLists) {
      if (list.parentId) {
        if (!childrenMap.has(list.parentId)) childrenMap.set(list.parentId, [])
        childrenMap.get(list.parentId)!.push(list.id)
      }
    }

    function collectDescendants(id: string): string[] {
      const ids = [id]
      const children = childrenMap.get(id) || []
      for (const childId of children) {

```

```

        ids.push(...collectDescendants(childId))
      }
      return ids
    }

    const allListIds = collectDescendants(filters.listId)
    where.lists = { some: { id: { in: allListIds } } }
  }

  if (filters.directListId) {
    where.lists = { some: { id: filters.directListId } }
  }

  if (filters.hasRating === 'true') {
    where.rating = { not: null }
  } else if (filters.hasRating === 'false') {
    where.rating = null
  }

  if (filters.search) {
    where.OR = [
      { title: { contains: filters.search, mode: 'insensitive' } },
      { note: { contains: filters.search, mode: 'insensitive' } },
      { url: { contains: filters.search, mode: 'insensitive' } },
      { platform: { contains: filters.search, mode: 'insensitive' } },
      { tags: { some: { name: { contains: filters.search, mode: 'insensitive' } } } },
      { lists: { some: { name: { contains: filters.search, mode: 'insensitive' } } } },
    ]
  }

  return where
}

/**
 * 构建排序条件，支持按评分排序且 null 始终在最后
 */
function buildCollectionOrderBy(sortBy: string, sortOrder: string):
Prisma.CollectionOrderByWithRelationInput[] {
  const order: Prisma.SortOrder = sortOrder === 'asc' ? 'asc' : 'desc'

  if (sortBy === 'rating') {
    // 按评分排序：有评分的按评分值排序，无评分（null）始终在最后
    return [
      { rating: { sort: order, nulls: 'last' } },
    ]
  }

```

```
        { createdAt: 'desc' },
      ]
    }

    return [{ createdAt: order }]
  }

// 获取收藏 (Offset 分页 - 兼容旧接口)
router.get('/', authenticate, [
  query('page').optional().isInt({ min: 1 }),
  query('limit').optional().isInt({ min: 1, max: 2000 }),
  query('tagId').optional().isUUID(),
  query('listId').optional().isUUID(),

  query('directListId').optional().isUUID(),
  query('platform').optional().isString(),
  query('platforms').optional().isString(),
  query('search').optional().isString(),
  query('hasRating').optional().isIn(['true', 'false']),
  query('sortBy').optional().isIn(['createdAt', 'rating']),
  query('sortOrder').optional().isIn(['asc', 'desc']),
], async (req: AuthenticatedRequest, res) => {
  const errors = validationResult(req)
  if (!errors.isEmpty()) {
    return ErrorResponse(res, 400, CommonErrorCodes.VALIDATION_FAILED, errors.array())
  }

  const q = req.query as { page?: string; limit?: string; tagId?: string; listId?: string; directListId?:
string; platform?: string; platforms?: string; search?: string; hasRating?: string; sortBy?: string;
sortOrder?: string }
  const page = Number(q.page) || 1
  const limit = Number(q.limit) || 20
  const userId = req.user.id
  const sortBy = q.sortBy || 'createdAt'
  const sortOrder = q.sortOrder || 'desc'

  try {
    const where = await buildCollectionWhere(userId, {
      tagId: q.tagId,
      listId: q.listId,
      directListId: q.directListId,
      platform: q.platform,
      platforms: q.platforms,
      search: q.search,
```

```
      hasRating: q.hasRating,
    })

    const [collections, total] = await Promise.all([
      prisma.collection.findMany({
        where,
        include: {
          tags: { select: { id: true, name: true } },
          lists: { select: { id: true, name: true } },
        },
        orderBy: buildCollectionOrderBy(sortBy, sortOrder),
        skip: (page - 1) * limit,
        take: limit,
      }),
      prisma.collection.count({ where }),
    ])

    res.json({
      data: collections.map(sanitizeCollection),
      pagination: {
        page,
        limit,
        total,
        totalPages: Math.ceil(total / limit),
      },
    })
  } catch (error) {
    logger.error({ err: error instanceof Error ? error.message : String(error) }, '获取收藏错误')
    errorResponse(res, 500, CollectionErrorCodes.COLLECTION_FETCH_FAILED)
  }
})

// 获取收藏 v2 (游标分页 - 高性能)
router.get('/v2', authenticate, [
  query('cursor').optional().isUUID().withMessage('cursor 必须是有效的 UUID'),
  query('limit').optional().isInt({ min: 1, max: 100 }),
  query('tagId').optional().isUUID(),
  query('listId').optional().isUUID(),
  query('directListId').optional().isUUID(),
  query('platform').optional().isString(),
  query('platforms').optional().isString(),
  query('search').optional().isString(),
  query('hasRating').optional().isIn(['true', 'false']),
  query('sortBy').optional().isIn(['createdAt', 'rating']),
])
```

```
    query('sortOrder').optional().isIn(['asc', 'desc']),
  ], async (req: AuthenticatedRequest, res) => {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return ErrorResponse(res, 400, CommonErrorCodes.VALIDATION_FAILED, errors.array())
    }

    const q = req.query as { cursor?: string; limit?: string; tagId?: string; listId?: string; directListId?:
string; platform?: string; platforms?: string; search?: string; hasRating?: string; sortBy?: string;
sortOrder?: string }
    const cursor = q.cursor
    const limit = Math.min(Number(q.limit) || 40, 100)
    const userId = req.user.id
    const sortBy = q.sortBy || 'createdAt'
    const sortOrder = q.sortOrder || 'desc'

    try {
      const where = await buildCollectionWhere(userId, {
        tagId: q.tagId,
        listId: q.listId,
        directListId: q.directListId,
        platform: q.platform,
        platforms: q.platforms,
        search: q.search,
        hasRating: q.hasRating,
      })

      // 游标分页: 多取一条用于判断 hasMore
      const collections = await prisma.collection.findMany({
        where,
        include: {
          tags: { select: { id: true, name: true } },
          lists: { select: { id: true, name: true } },
        },
        orderBy: buildCollectionOrderBy(sortBy, sortOrder),
        cursor: cursor ? { id: cursor } : undefined,
        skip: cursor ? 1 : 0,
        take: limit + 1,
      })

      const hasMore = collections.length > limit
      const data = hasMore ? collections.slice(0, limit) : collections
      const nextCursor = hasMore && data.length > 0 ? data[data.length - 1].id : undefined
    }
  })
}
```

```
    res.json({
      data: data.map(sanitizeCollection),
      pagination: {
        limit,
        hasMore,
        nextCursor,
      },
    })
  } catch (error) {
    logger.error({ err: error instanceof Error ? error.message : String(error) }, '获取收藏 V2 错误')
    errorResponse(res, 500, CollectionErrorCodes.COLLECTION_FETCH_FAILED)
  }
})

// 添加收藏
router.post('/', authenticate, [
  body('url').custom((value) => {
    const processed = ensureHttps(value)
    return !!processed && isURL(processed)
  }).withMessage('请输入有效的 URL'),
  body('title').isLength({ min: 1, max: 200 }).withMessage('标题不能为空且不超过 200 字符'),
  body('note').optional().isLength({ max: 100 }).withMessage('备注不超过 100 字符'),
  body('rating').optional().isFloat({ min: 0.5, max: 5 }).withMessage('评分需在 0.5-5 之间'),
  body('tagIds').optional().isArray(),
  body('listIds').optional().isArray({ max: 1 }).withMessage('一个收藏只能属于一个分组'),
  body('coverStrategy').optional().isIn(['url', 'brand', 'ai']).withMessage('封面策略只能是 url、brand 或 ai'),
  body('pageType').optional().isIn(['home', 'detail', 'list', 'search', 'navigation', 'document', 'download', 'other']).withMessage('页面类型不正确'),
], async (req: AuthenticatedRequest, res) => {
  const errors = validationResult(req)
  if (!errors.isEmpty()) {
    return errorResponse(res, 400, CommonErrorCodes.VALIDATION_FAILED, errors.array())
  }

  const { url, title, coverImage, note, rating, tagIds = [], listIds = [], coverStrategy, pageType } = req.body

  const userId = req.user.id
  const platform = detectPlatform(url)

  try {
    // 配额检查
```

```
}
```

```
// ===== API 响应 =====
```

```
export interface ApiResponse<T> {  
  data: T;  
  pagination?: {  
    page: number;  
    limit: number;  
    total: number;  
    totalPages: number;  
  };  
}
```

```
// ===== Admin 类型 =====
```

```
export interface LogQueryParams {  
  level?: string;  
  startTime?: string;  
  endTime?: string;  
  keyword?: string;  
  errorCode?: string;  
  path?: string;  
  page?: number;  
  pageSize?: number;  
}
```

```
export interface ErrorQueryParams {  
  status?: string;  
  errorCode?: string;  
  page?: number;  
  pageSize?: number;  
  sortBy?: string;  
  sortOrder?: string;  
}
```

```
export interface AlertRuleInput {  
  name: string;  
  type: 'error_rate' | 'error_count' | 'response_time' | 'service_down';  
  conditionConfig: Record<string, number>;  
  channels?: Record<string, string[]>;  
  enabled?: boolean;  
  cooldownMinutes?: number;  
  priority?: 'P0' | 'P1' | 'P2' | 'P3';  
}
```

```
    silentStart?: string | null;
    silentEnd?: string | null;
  }

export interface UserQueryParams {
  page?: number;
  pageSize?: number;
  keyword?: string;
  sortBy?: string;
  sortOrder?: string;
}

export interface TierConfigInput {
  key: string;
  nameZh: string;
  nameEn: string;
  description?: string;
  sortOrder?: number;
  isActive?: boolean;
  quotaConfig: Record<string, number>;
  pricingConfig?: Record<string, unknown> | null;
  benefits?: string[];
}

import axios from 'axios';
import { getErrorMessage } from '@linkchest/i18n';
import { getToken, logout } from '../auth';

const isBrowser = typeof window !== 'undefined';
const API_BASE_URL = isBrowser
  ? '/api'
  : (process.env.NEXT_PUBLIC_API_URL || 'http://localhost:3001/api');
const PUBLIC_BASE_URL = isBrowser
  ? ''
  : API_BASE_URL.replace(/\/api$/, '');

export const api = axios.create({
  baseURL: API_BASE_URL,
  timeout: 60000,
  headers: {
    'Content-Type': 'application/json',
  },
});
```



```
export const publicApi = axios.create({
  baseURL: PUBLIC_BASE_URL,
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
  },
});

// 请求拦截器
api.interceptors.request.use(
  (config) => {
    const token = getToken();
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// 响应拦截器
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.data && typeof error.response.data === 'object') {
      const rawLocale = typeof window !== 'undefined' ?
localStorage.getItem('linkchest-locale') : 'en';
      const locale = rawLocale === 'zh' || rawLocale === 'en' || rawLocale === 'ja' || rawLocale
=== 'ko' || rawLocale === 'fr' || rawLocale === 'de' ? rawLocale : 'en';
      const errCode = error.response.data.error;
      if (typeof errCode === 'string') {
        error.response.data.message = getErrorMessage(errCode, locale);
      }
    }

    if (error.response?.status === 401) {
      const isAuthRequest = error.config?.url?.includes('/auth/');
      const isAdminRequest = error.config?.url?.includes('/admin/');
      if (!isAuthRequest && !isAdminRequest) {
        logout();
      }
    } else if (error.response?.status === 403) {
      const errCode = error.response.data?.error || '';
      if (typeof errCode === 'string' && errCode.startsWith('QUOTA_EXCEEDED')) {

```

```
        if (typeof window !== 'undefined') {
            window.dispatchEvent(new CustomEvent('quota-exceeded', { detail: errCode }));
        }
    }
} else if (error.response?.status === 429) {
    const message = error.response.data?.message || '请求过于频繁，请稍后再试';
    if (typeof window !== 'undefined') {
        window.dispatchEvent(new CustomEvent('rate-limited', { detail: message }));
    }
}
return Promise.reject(error);
}
);

export interface ApiError {
    response?: { status?: number; data?: { error?: string; message?: string; errors?: { msg?: string }[]; needPassword?: boolean; [key: string]: unknown } };
    message?: string;
    statusCode?: number;
}

'use client';

import { useState, useEffect, useCallback, useRef } from 'react';
import { useVirtualizer } from '@tanstack/react-virtual';
import { useQuery, useInfiniteQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { useSearchParams } from 'next/navigation';
import { Search, X, LayoutGrid, LayoutTemplate, Filter, ChevronDown, Tag as TagIcon, Move, Check, CheckSquare, MinusCircle, ArrowLeft, XCircle, Archive, Inbox, Loader2, FolderOpen, Trash2, Edit2, ExternalLink, ArrowUpDown, Settings } from 'lucide-react';
import { api } from '@lib/api';
import Link from 'next/link';
import { getToken } from '@lib/auth';
import { useI18n, getListDisplayName, getListPathDisplayName } from '@lib/i18n';
import { platformNames, getContrastTextColor, PLATFORMS, generateDefaultCover } from '@lib/platforms';
import { PAGE_TYPES, PageTypeIcon, getPageTypeConfig } from '@lib/pageTypes';
import LazyImage from './LazyImage';
import UndoToast from './UndoToast';
import { EmptyState, CollectionSkeletonGrid, CollectionSkeletonList } from './ui';
import StarRating from './StarRating';
import dynamic from 'next/dynamic';
```

```
const CollectionDetailModal = dynamic(() => import('./CollectionDetailModal'), { ssr: false });
import { useToast } from './Toast';
import type { DisplayFieldKey, CollectionViews } from
'@/app/(main)/settings/CollectionViewConfig';
import { useLocalCollectionViews } from '@/hooks/useLocalCollectionViews';

interface Collection {
  id: string;
  title: string;
  coverImage: string | null;
  platform: string;
  pageType?: string;
  note: string | null;
  rating?: number | null;
  tags: { id: string; name: string }[];
  lists: { id: string; name: string; isDefault?: boolean }[];
  url: string;
  createdAt?: string;
}

// 获取排序后的字段配置
function getSortedFields(views: CollectionViews | undefined, mode: 'webGrid' | 'webList'): { key:
DisplayFieldKey; enabled: boolean }[] {
  if (!views?.[mode]?.fields) {
    return [
      { key: 'cover', enabled: true },
      { key: 'title', enabled: true },
      { key: 'platform', enabled: true },
      { key: 'rating', enabled: true },
      { key: 'pageType', enabled: false },
      { key: 'tags', enabled: true },

      { key: 'lists', enabled: true },
      { key: 'note', enabled: true },
      { key: 'createdAt', enabled: false },
    ];
  }
  return [...views[mode].fields]
    .sort((a, b) => a.order - b.order)
    .map(f => ({ key: f.key, enabled: f.enabled }));
}

export default function CollectionList() {
  const queryClient = useQueryClient();
```

```
const searchParams = useSearchParams();
const { t } = useI18n();
const { showToast, showAlert } = useToast();
const [searchQuery, setSearchQuery] = useState('');
const [debouncedSearch, setDebouncedSearch] = useState('');
const [toast, setToast] = useState<{ id: string; message: string; restoreData: Collection } | null>(null);

const [viewMode, setViewMode] = useState<'grid' | 'card'>('grid');
const [showFilters, setShowFilters] = useState(false);
const [filterPlatforms, setFilterPlatforms] = useState<Set<string>>(new Set());
const [filterTagIds, setFilterTagIds] = useState<Set<string>>(new Set());
const [filterListIds, setFilterListIds] = useState<Set<string>>(new Set());
const [filterHasRating, setFilterHasRating] = useState<boolean | null>(null);
const [filterPageType, setFilterPageType] = useState<string>('');
const [sortBy, setSortBy] = useState<'createdAt' | 'rating'>('createdAt');
const [sortOrder, setSortOrder] = useState<'asc' | 'desc'>('desc');
const [moveModal, setMoveModal] = useState<{ type: 'list' | 'tag'; item: Collection } | null>(null);
const [detailItem, setDetailItem] = useState<Collection | null>(null);
const [showSortModal, setShowSortModal] = useState(false);

// 视图配置（本地存储）
const { views: collectionViews } = useLocalCollectionViews();
const viewConfig = getSortedFields(collectionViews, viewMode === 'grid' ? 'webGrid' : 'webList');
const enabledFields = new Set(viewConfig.filter(f => f.enabled).map(f => f.key));

// URL 参数筛选：从分组页/标签页点击跳转
const [urlFilterLabel, setUrlFilterLabel] = useState('');
const [urlFilterType, setUrlFilterType] = useState<'list' | 'tag' | null>(null);
const [urlFilterId, setUrlFilterId] = useState('');

// 批量操作状态
const [editMode, setEditMode] = useState(false);
const [selectedIds, setSelectedIds] = useState<Set<string>>(new Set());
const [batchModal, setBatchModal] = useState<'list' | 'tag' | null>(null);

// 滚动位置记忆 + 自动加载更多
const listContainerRef = useRef<HTMLDivElement>(null);
const loadMoreRef = useRef<HTMLDivElement>(null);

useEffect(() => {
  // 读取 URL 参数
  const listId = searchParams.get('listId');
```

```
const listName = searchParams.get('listName');
const tagId = searchParams.get('tagId');
const tagName = searchParams.get('tagName');
if (listId) {
  setFilterListIds(new Set([listId]));
  setUrlFilterId(listId);
  setUrlFilterType('list');
  setUrlFilterLabel(listName || t('sidebar.groups'));
} else if (tagId) {
  setFilterTagIds(new Set([tagId]));
  setUrlFilterId(tagId);
  setUrlFilterType('tag');
  setUrlFilterLabel(`#${tagName} || t('collection.tag')`);
}
// 恢复滚动位置
const savedScroll = sessionStorage.getItem('collectionListScroll');
if (savedScroll && listContainerRef.current) {
  listContainerRef.current.scrollTop = parseInt(savedScroll, 10);
}
}, [searchParams]);

// 记忆滚动位置
useEffect(() => {
  const container = listContainerRef.current;
  if (!container) return;
  const handleScroll = () => {
    sessionStorage.setItem('collectionListScroll', String(container.scrollTop));
  };
  container.addEventListener('scroll', handleScroll, { passive: true });
  return () => container.removeEventListener('scroll', handleScroll);
}, []);

// 搜索防抖
useEffect(() => {
  const timer = setTimeout(() => {
    setDebounceSearch(searchQuery);
  }, 300);
  return () => clearTimeout(timer);
}, [searchQuery]);

// Toast 自动消失
useEffect(() => {
  if (toast) {
    const timer = setTimeout(() => setToast(null), 5000);
  }
}
```

```
        return () => clearTimeout(timer);
    }
}, [toast]);

// 无限滚动查询: 筛选条件传给后端, 分页懒加载
const {

    data,
    isLoading,
    isError,
    error,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
    refetch,
} = useInfiniteQuery({
    queryKey: ['collections', Array.from(filterTagIds).sort().join(','),
Array.from(filterListIds).sort().join(','), debouncedSearch, Array.from(filterPlatforms).sort().join(','),
filterHasRating, filterPageType, sortBy, sortOrder],
    queryFn: async ({ pageParam = 1 }) => {
        const params = new URLSearchParams();
        params.set('page', String(pageParam));
        params.set('limit', '40');
        if (filterTagIds.size > 0) params.set('tagIds', Array.from(filterTagIds).join(','));
        if (filterListIds.size > 0) params.set('listIds', Array.from(filterListIds).join(','));
        if (debouncedSearch) params.set('search', debouncedSearch);
        if (filterPlatforms.size > 0) params.set('platforms', Array.from(filterPlatforms).join(','));
        if (filterHasRating !== null) params.set('hasRating', filterHasRating ? 'true' : 'false');
        if (filterPageType) params.set('pageType', filterPageType);
        if (sortBy !== 'createdAt') params.set('sortBy', sortBy);
        if (sortOrder !== 'desc') params.set('sortOrder', sortOrder);
        const response = await api.get(`/collections?${params.toString()}`);
        return response.data;
    },
    getNextPageParam: (lastPage) => {
        const { page, totalPages } = lastPage.pagination;
        return page < totalPages ? page + 1 : undefined;
    },
    initialPageParam: 1,
    enabled: !!getToken(),
    staleTime: 30 * 1000,
});

// Tags & Lists for filter dropdowns
```

```
interface TagItem { id: string; name: string; }
interface ListItem { id: string; name: string; isDefault?: boolean; depth?: number; }

const { data: tagsData } = useQuery({
  queryKey: ['tags'],
  queryFn: async () => { const r = await api.get('/tags'); return (r.data.data || r.data) as TagItem[]; },
  enabled: !!getToken(),
});

const { data: listsData } = useQuery({
  queryKey: ['lists', 'flat'],
  queryFn: async () => { const r = await api.get('/lists/flat'); return (r.data.data || r.data) as ListItem[]; },
  enabled: !!getToken(),
});

const allCollections: Collection[] = data?.pages.flatMap((page: { data: Collection[] }) =>

  (page.data || []).map((item: Collection) => ({
    ...item,
    tags: item.tags || [],
    lists: item.lists || [],
  }))
) || [];

const totalCount = data?.pages[0]?.pagination?.total || 0;
const hasMore = hasNextPage;

// 虚拟滚动：跟踪容器宽度以计算每行列数
const [containerWidth, setContainerWidth] = useState(0);
useEffect(() => {
  const el = listContainerRef.current;
  if (!el) return;
  const ro = new ResizeObserver(entries => {
    for (const e of entries) setContainerWidth(e.contentRect.width);
  });
  ro.observe(el);
  setContainerWidth(el.clientWidth);
  return () => ro.disconnect();
}, []);

// 自动加载更多（IntersectionObserver）
useEffect(() => {
  if (!loadMoreRef.current || !hasNextPage || isFetchingNextPage) return;
```

```
const observer = new IntersectionObserver(
  (entries) => {
    if (entries[0].isIntersecting) {
      fetchNextPage();
    }
  },
  { root: listContainerRef.current, rootMargin: '200px' }
);
observer.observe(loadMoreRef.current);
return () => observer.disconnect();
}, [hasNextPage, isFetchingNextPage, fetchNextPage]);

// 后端已处理筛选，前端直接使用聚合结果
const collections = allCollections;

// 虚拟滚动：按行虚拟化，每行 2 列（desktop）
const COLS = containerWidth >= 1024 ? 2 : 1;
const ROW_HEIGHT = 156; // 虚拟滚动行高
const ITEM_HEIGHT = 140; // 实际 item 高度，小于 ROW_HEIGHT 形成竖直间距
const virtualizer = useVirtualizer({
  count: Math.ceil(collections.length / COLS),
  getScrollElement: () => listContainerRef.current,
  estimateSize: () => ROW_HEIGHT,
  overscan: 5,
});

// 批量操作
const batchDeleteMutation = useMutation({

  mutationFn: (ids: string[]) => api.post('/collections/batch-delete', { ids }),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['collections'] });
    setSelectedIds(new Set());
    setEditMode(false);
    showToast(t('edit.deleteSuccess'), 'success');
  },
});

const toggleSelect = (id: string) => {
  const newSet = new Set(selectedIds);
  if (newSet.has(id)) newSet.delete(id); else newSet.add(id);
  setSelectedIds(newSet);
};
```



```
const selectAll = () => setSelectedIds(new Set(collections.map(c => c.id)));
const clearSelection = () => setSelectedIds(new Set());

const clearUrlFilter = () => {
  if (urlFilterType === 'list') setFilterListIds(new Set());
  else if (urlFilterType === 'tag') setFilterTagIds(new Set());
  setUrlFilterType(null);
  setUrlFilterLabel('');
  setUrlFilterId('');
  // 清除 URL 参数
  if (typeof window !== 'undefined') {
    window.history.replaceState({}, '', '/');
  }
};

// 删除收藏（带撤销）
const handleDelete = useCallback(async (item: Collection) => {
  if (!confirm(t('collection.deleteConfirm'))) return;
  try {
    await api.delete(`/collections/${item.id}`);
    queryClient.invalidateQueries({ queryKey: ['collections'] });
    setDetailItem(null);
    setToast({
      id: item.id,
      message: t('collection.deleted', { title: item.title }),
      restoreData: item,
    });
  } catch {
    showAlert(t('common.operationFailed'), 'error');
  }
});

// 认证工具函数，token 同时存储在 cookie 和 localStorage 中
// cookie 供 Next.js Middleware 读取，localStorage 供 axios 拦截器读取

const TOKEN_KEY = 'linkchest_token';
const USER_KEY = 'linkchest_user';

const COOKIE_MAX_AGE = 30 * 24 * 60 * 60; // 30 天

function setCookie(name: string, value: string, maxAge: number): void {
  if (typeof document === 'undefined') return;
  const secure = location.protocol === 'https:' ? 'Secure' : '';
  // Partitioned 增强隐私隔离（CHIPS），防止跨上下文共享 cookie
  document.cookie = `${name}=${encodeURIComponent(value)}; Path=/; Max-Age=${maxAge}; SameSite=Lax${secure}; Partitioned`;
}
```

```
}

function getCookie(name: string): string | null {
  if (typeof document === 'undefined') return null;
  const match = document.cookie.match(new RegExp(`(?:^|; )${name}=[^;]*`));
  return match ? decodeURIComponent(match[1]) : null;
}

function deleteCookie(name: string): void {
  if (typeof document === 'undefined') return;
  const secure = location.protocol === 'https:' ? 'Secure' : '';
  const pastDate = 'Thu, 01 Jan 1970 00:00:00 GMT';
  // 同时设置 Expires 和 Max-Age 确保所有浏览器兼容, Partitioned 属性必须与设置时一致
  document.cookie = `${name}=; Path=/; Expires=${pastDate}; Max-Age=0; SameSite=Lax${secure}; Partitioned`;
}

export function getToken(): string | null {
  if (typeof window === 'undefined') return null;
  // 优先从 cookie 读取, 兼容 localStorage 旧数据
  return getCookie(TOKEN_KEY) || localStorage.getItem(TOKEN_KEY);
}

export function setToken(token: string): void {
  // 双写: cookie + localStorage
  localStorage.setItem(TOKEN_KEY, token);
  setCookie(TOKEN_KEY, token, COOKIE_MAX_AGE);
}

export function removeToken(): void {
  localStorage.removeItem(TOKEN_KEY);
  deleteCookie(TOKEN_KEY);
}

export function getUser(): Record<string, unknown> | null {
  if (typeof window === 'undefined') return null;
  const userStr = localStorage.getItem(USER_KEY);

  return userStr ? JSON.parse(userStr) : null;
}

export function setUser(user: Record<string, unknown>): void {
  localStorage.setItem(USER_KEY, JSON.stringify(user));
}
```

```
// 通知所有监听用户变化的组件（如 Sidebar）
if (typeof window !== 'undefined') {
  window.dispatchEvent(new CustomEvent('linkchest-user-updated', { detail: user }));
}
}

export function removeUser(): void {
  localStorage.removeItem(USER_KEY);
}

export function isLoggedIn(): boolean {
  return !!getToken();
}

export function logout(): void {
  removeToken();
  removeUser();
  window.dispatchEvent(new CustomEvent('linkchest-logout'));
  // 使用 replace 避免在历史记录中留下当前受保护页面
  // 添加 logout 参数 + 时间戳: 1) 跳过 login 页面的 isLoggedIn 自动重定向 2) 防止缓存 3) 强制完整页面刷新
  window.location.replace('/login?logout=1&t=' + Date.now());
}

'use client'

import React, { useEffect, useState, useRef, useCallback } from 'react'
import { useRouter, useParams, useSearchParams } from 'next/navigation'
import { useI18n } from '@lib/i18n'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '@lib/api'
import { getPlatformName, getPlatformColor, getPlatformIcon, isPlatformValid } from '@lib/platforms'
import { isValidUrl, parseShareText, parseUrlPlatform } from '@lib/utills'
import CoverEditor from '@components/CoverEditor'
import StarRating from '@components/StarRating'
import { PAGE_TYPES, DEFAULT_PAGE_TYPE, getPageTypeConfig, PageTypeIcon } from '@lib/pageTypes'

type CollectionFormMode = 'add' | 'edit'

interface Tag {
  id: string
  name: string
}
```

```
    collectionCount?: number
  }
```

```
interface ListItem {
  id: string
  name: string
  parentId: string | null
  collectionCount: number
  totalCollectionCount?: number
  isDefault?: boolean
  depth?: number
  path?: { id: string; name: string; isDefault?: boolean }[]
  pathName?: string | null
  hasChildren?: boolean
  children?: ListItem[]
}
```

```
interface Collection {
  id: string
  url: string
  title: string
  coverImage?: string
  platform: string
  note?: string
  tags: Tag[]
  lists: ListItem[]
  rating?: number | null
  pageType?: string
  createdAt: string
}
```

```
interface Props {
  mode: CollectionFormMode
  preselectedTagId?: string

  preselectedListId?: string
}
```

```
export default function CollectionForm({ mode, preselectedTagId, preselectedListId }: Props) {
  const router = useRouter()
  const params = useParams()
  const searchParams = useSearchParams()
  const { t } = useI18n()
  const queryClient = useQueryClient()
```

```
const isAdd = mode === 'add'
const isEdit = mode === 'edit'
const collectionId = isEdit ? (params.id as string) : undefined

const initialUrl = searchParams?.get('url') || ''

// 表单状态
const [url, setUrl] = useState('')
const [title, setTitle] = useState('')
const [coverImage, setCoverImage] = useState('')
const [platform, setPlatform] = useState('other')
const [note, setNote] = useState('')
const [selectedTags, setSelectedTags] = useState<string[]>(  
  isAdd ? (preselectedTagId ? [preselectedTagId] : []) : []  
)
const [selectedListIds, setSelectedListIds] = useState<string[]>(  
  preselectedListId ? [preselectedListId] : []  
)
const [selectedPageType, setSelectedPageType] = useState<string>(DEFAULT_PAGE_TYPE)
const [rating, setRating] = useState<number | null>(null)
const [expandedListIds, setExpandedListIds] = useState<Set<string>>(new Set())

// UI 状态
const [tagSectionExpanded, setTagSectionExpanded] = useState(false)
const [groupSectionExpanded, setGroupSectionExpanded] = useState(false)
const [pageTypeSectionExpanded, setPageTypeSectionExpanded] = useState(false)

const [newTagModalVisible, setNewTagModalVisible] = useState(false)
const [newTagName, setNewTagName] = useState('')
const [newListModalVisible, setNewListModalVisible] = useState(false)
const [newListName, setNewListName] = useState('')

const [parsing, setParsing] = useState(false)
const [parseError, setParseError] = useState('')
const [parsePhase, setParsePhase] = useState('')

const [duplicateWarning, setDuplicateWarning] = useState<any>(null)
const [titleDuplicateWarning, setTitleDuplicateWarning] = useState<any>(null)

const titleCheckTimer = useRef<NodeJS.Timeout | null>(null)

// 获取标签列表
const { data: tagsData } = useQuery({
```

```
    queryKey: ['tags'],
    queryFn: async () => {
      const response = await api.get('/tags')
      return response.data.data || []
    },
  })

// 获取分组列表（扁平列表）
const { data: listsData } = useQuery({
  queryKey: ['lists', 'flat'],
  queryFn: async () => {
    const response = await api.get('/lists/flat')
    return response.data.data || []
  },
})

// 编辑模式：加载已有数据
const { data: collection, isLoading: isLoadingCollection } = useQuery({
  queryKey: ['collection', collectionId],
  queryFn: async () => {
    if (!collectionId) return null
    const response = await api.get(`/collections/${collectionId}`)
    return response.data.data || response.data
  },
  enabled: isEdit && !!collectionId,
})

// 新增模式：读取用户设置
const { data: userSettings } = useQuery({
  queryKey: ['users', 'settings'],
  queryFn: async () => {
    if (isEdit) return null
    const response = await api.get('/users/settings')
    return response.data?.data
  },
  enabled: isAdd,
})

useEffect(() => {
  if (isEdit && collection) {
    setUrl(collection.url || '')
    setTitle(collection.title || '')
    setCoverImage(collection.coverImage || '')
    setPlatform(collection.platform || 'other')
  }
})
```

```
      setNote(collection.note || '')
      setSelectedTags(collection.tags?.map((tg: Tag) => tg.id) || [])
      setSelectedListIds(collection.lists?.map((l: ListItem) => l.id) || [])
      setRating(collection.rating ?? null)
      setSelectedPageType(collection.pageType || DEFAULT_PAGE_TYPE)
    }
  }, [isEdit, collection])

  useEffect(() => {
    if (userSettings && !preselectedListId && !preselectedTagId) {
      if (userSettings.defaultListId) {
        setSelectedListIds([userSettings.defaultListId])
      }
      if (userSettings.defaultTagIds?.length) {
        setSelectedTags(userSettings.defaultTagIds)
      }
    }
  }, [userSettings, preselectedListId, preselectedTagId])

  useEffect(() => {
    if (listsData && selectedListIds.length === 0) {
      const defaultList = listsData.find((l: any) => l.isDefault)
      if (defaultList) {
        setSelectedListIds([defaultList.id])
      } else if (listsData.length > 0) {
        setSelectedListIds([listsData[0].id])
      }
    }
  }, [listsData, selectedListIds.length])

  useEffect(() => {
    if (isAdd && initialUrl) {
      setUrl(initialUrl)
      parseUrl(initialUrl, true)
    }
  }, [isAdd, initialUrl])

  // 新增收藏
  const createMutation = useMutation({
    mutationFn: async (data: any) => {
      const response = await api.post('/collections', data)
      return response.data.data || response.data
    },
  })
```

```
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ['collections'] })
        queryClient.invalidateQueries({ queryKey: ['lists'] })
        queryClient.invalidateQueries({ queryKey: ['quota'] })
        router.push('/')
      },
      onError: (error: any) => {
        console.error('Create failed:', error)
      }
    }
  },
  'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { useState, useEffect } from 'react';
import {
  FolderOpen,
  Share2,
  Settings,
  Sun,
  Moon,
  LogOut,
  Plus,
  ChevronRight,
  Tag,
  LayoutGrid,
  PanelLeftClose,
  PanelLeftOpen,
  X,
  Trash2,
  SlidersHorizontal,
} from 'lucide-react';
import { getUser, setUser as saveUser, logout, isLoggedIn } from '@lib/auth';
import { api } from '@lib/api';
import { useTheme } from '@hooks/useTheme';
import { useI18n } from '@lib/i18n';
import { cn } from '@lib/Utils';
import Logo from './Logo';

interface SidebarProps {
  collapsed?: boolean;
  onToggle?: () => void;
  mobile?: boolean;
  onMobileClose?: () => void;
}
```



```
export default function Sidebar({
  collapsed = false,
  onToggle,
  mobile = false,
  onMobileClose,
}: SidebarProps) {
  const pathname = usePathname();
  const [user, setUser] = useState<any>(null);
  const [mounted, setMounted] = useState(false);
  const { theme, toggleTheme } = useTheme();
  const { t } = useI18n();

  const menuItems = [
    { icon: LayoutGrid, label: t('sidebar.collections'), href: '/collections' },
    { icon: SlidersHorizontal, label: t('sidebar.manage'), href: '/manage' },

    { icon: Settings, label: t('sidebar.settings'), href: '/settings' },
  ];

  useEffect(() => {
    const stored = getUser();
    setUser(stored);
    setMounted(true);

    if (isLoggedIn()) {
      api
        .get('/auth/me')
        .then((res) => {
          const userData = res.data.data || res.data;
          if (userData && userData.id) {
            saveUser(userData);
            setUser(userData);
          }
        })
        .catch(() => {
          /* ignore */
        });
    }
  });

  const handleUserUpdate = (e: Event) => {
    const userData = (e as CustomEvent).detail;
    if (userData && userData.id) {
      setUser(userData);
    }
  };
}
```

```

    }
  };
  window.addEventListener('linkchest-user-updated', handleUserUpdate);
  return () =>
    window.removeEventListener('linkchest-user-updated', handleUserUpdate);
}, []);

if (!mounted) {
  return (
    <aside className="w-full h-full flex flex-col bg-chest-500 border-r border-chest-400/20">
      <div
        className={cn(
          'py-5 border-b border-parchment/10',
          collapsed ? 'px-2 flex justify-center' : 'px-5'
        )}
      >
        <div className="flex items-center gap-3">
          <div className="w-10 h-10 rounded-md bg-amber-400 flex items-center justify-center">
            <LayoutGrid size={20} className="text-chest-500" />
          </div>
          <{!collapsed && (
            <div>
              <h1 className="text-lg font-bold text-parchment leading-tight">

                {t('sidebar.appName')}

              </h1>
              <p className="text-xs text-parchment/50 mt-0.5">
                {t('sidebar.subtitle')}
              </p>
            </div>
          )}
        </div>
      </div>
      <div className="flex-1" />
    </aside>
  );
}

return (
  <aside className="w-full h-full flex flex-col bg-chest-500 border-r border-chest-400/20 shadow-[4px_0_24px_rgba(0,0,0,0.08)] transition-colors duration-200">
    { /* Mobile Close Button */ }
    {mobile && onMobileClose && (

```

```

        <div className="flex items-center justify-end px-4 py-2 border-b border-parchment/10 md:hidden">
            <button
                onClick={onMobileClose}
                className="p-1.5 rounded-md text-parchment/60 hover:bg-parchment/10 transition-colors"
            >
                <X size={20} />
            </button>
        </div>
    )}

    { /* Logo 区域 */ }
    <div
        className={cn(
            'py-5 border-b border-parchment/10',
            collapsed ? 'px-2' : 'px-5'
        )}
    >
        <div
            className={cn(
                'flex items-center',
                collapsed ? 'justify-center' : 'gap-3'
            )}
        >
            <Logo size={collapsed ? 32 : 40} variant="dark" />
            { !collapsed && (
                <div>
                    <h1 className="text-lg font-display font-semibold text-parchment leading-tight tracking-tight">
                        {t('sidebar.appName')}
                    </h1>
                    <p className="text-xs text-parchment/50 mt-0.5 font-sans">
                        {t('sidebar.subtitle')}
                    </p>
                </div>
            ) }
        </div>
    >
    </div>

    { /* 添加收藏按钮 */ }
    <div className={cn('pt-4', collapsed ? 'px-2' : 'px-4')}>
        <Link

```

```

      href="/add"
      className={cn(
        'flex items-center justify-center gap-2 w-full bg-amber-400 text-chest-500
rounded-md font-semibold text-sm shadow-elevated hover:bg-amber-500
hover:shadow-card-hover hover:scale-[1.02] active:scale-[0.98] active:shadow-card transition-all
duration-200',
        collapsed ? 'px-2 py-3' : 'px-4 py-3'
      )}
    >
    <Plus size={18} strokeWidth={2.5} />
    {!collapsed && <span>{t('sidebar.addCollection')}</span>}
  </Link>
</div>

```

```

{ /* 导航菜单 */}
<nav
  className={cn(
    'flex-1 py-4 space-y-0.5',
    collapsed ? 'px-2' : 'px-3'
  )}
>
  {menuItems.map((item) => {
    const Icon = item.icon;
    const isActive = pathname === item.href || (item.href === '/manage' &&
pathname?.startsWith('/manage')) || (item.href === '/collections' && (pathname ===
'/collections' || pathname === '/'));

    return (
      <Link
        key={item.href}
        href={item.href}
        onClick={() => mobile && onMobileClose?.()}
        className={cn(
          'group relative flex items-center rounded-md text-sm font-medium
transition-all duration-200',
          isActive
            ? 'bg-parchment/10 text-amber-300'
            : 'text-parchment/60 hover:bg-parchment/5 hover:text-parchment/90
hover:translate-x-0.5',
          collapsed ? 'justify-center px-2 py-2.5' : 'gap-3 px-3 py-2.5'
        )}
      >
        {isActive && !collapsed && (
          <span className="absolute left-0 top-1/2 -translate-y-1/2 w-[3px] h-5

```

```
rounded-r-full bg-amber-400 shadow-[0_0_8px_rgba(200,149,108,0.4)]" />
    ))
    <Icon
      size={18}
      strokeWidth={isActive ? 2.5 : 2}

'use client';

import { createContext, useContext, useState, useCallback, useEffect, ReactNode } from 'react';
import { X, CheckCircle, AlertCircle, Info } from 'lucide-react';

type ToastType = 'success' | 'error' | 'info';

type ToastMode = 'toast' | 'alert';

interface ToastItem {
  id: number;
  message: string;
  type: ToastType;
  mode: ToastMode;
}

interface ToastContextType {
  showToast: (message: string, type?: ToastType) => void;
  showAlert: (message: string, type?: ToastType) => void;
}

const ToastContext = createContext<ToastContextType | null>(null);

let toastId = 0;

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<ToastItem[]>([]);

  const showToast = useCallback((message: string, type: ToastType = 'info') => {
    const id = ++toastId;
    setToasts(prev => [...prev, { id, message, type, mode: 'toast' }]);
  }, []);

  const showAlert = useCallback((message: string, type: ToastType = 'info') => {
    const id = ++toastId;
    setToasts(prev => [...prev, { id, message, type, mode: 'alert' }]);
  }, []);
```

```

const removeToast = useCallback((id: number) => {
  setToasts(prev => prev.filter(t => t.id !== id));
}, []);

return (
  <ToastContext.Provider value={{ showToast, showAlert }}>
    {children}
    <ToastContainer toasts={toasts} removeToast={removeToast} />
  </ToastContext.Provider>
);
}

function ToastContainer({ toasts, removeToast }: { toasts: ToastItem[]; removeToast: (id: number) => void }) {

  const alerts = toasts.filter(t => t.mode === 'alert');
  const toastItems = toasts.filter(t => t.mode === 'toast');

  return (
    <>
      /* Alert 模式 - 全屏模态弹窗 */
      {alerts.length > 0 && (
        <div className="fixed inset-0 z-[100] flex items-center justify-center bg-black/40 backdrop-blur-sm">
          <div className="flex flex-col gap-3 max-w-sm w-full mx-4">
            {alerts.map(toast => (
              <AlertItem key={toast.id} toast={toast} onClose={() => removeToast(toast.id)} />
            ))}
          </div>
        </div>
      )}

      /* Toast 模式 - 底部轻提醒 */
      {toastItems.length > 0 && (
        <div className="fixed bottom-8 left-0 right-0 z-[100] flex flex-col items-center gap-2 pointer-events-none px-4">
          {toastItems.map(toast => (
            <ToastItemComponent key={toast.id} toast={toast} onClose={() => removeToast(toast.id)} />
          ))}
        </div>
      )}
    </>
  );
}

```

```

}

const icons = {
  success: <CheckCircle size={20} className="text-sage flex-shrink-0" />,
  error: <AlertCircle size={20} className="text-rust flex-shrink-0" />,
  info: <Info size={20} className="text-chest-500 dark:text-amber-400 flex-shrink-0" />,
};

const bgColors = {
  success: 'border-sage/20 dark:border-sage/30 bg-sage/10 dark:bg-sage/20',
  error: 'border-rust/20 dark:border-rust/30 bg-rust/10 dark:bg-rust/20',
  info: 'border-chest-500/15 dark:border-amber-400/20 bg-chest-500/5 dark:bg-amber-400/10',
};

function AlertItem({ toast, onClose }: { toast: ToastItem; onClose: () => void }) {
  return (
    <div
      className={`flex items-center gap-3 px-5 py-4 bg-white dark:bg-chest-800/90
rounded-2xl shadow-2xl border ${bgColors[toast.type]} animate-scale-in pointer-events-auto`}
    >
      {icons[toast.type]}
      <p
        className="flex-1 text-base font-medium text-charcoal
dark:text-parchment">{toast.message}</p>

      <button
        onClick={onClose}
        className="p-1.5 hover:bg-parchment/20
dark:hover:bg-chest-700/40 rounded-lg transition-colors cursor-pointer">
        <X size={18} className="text-taupe" />
      </button>
    </div>
  );
}

function ToastItemComponent({ toast, onClose }: { toast: ToastItem; onClose: () => void }) {
  useEffect(() => {
    const timer = setTimeout(() => {
      onClose();
    }, 3000);
    return () => clearTimeout(timer);
  }, [onClose]);

  return (
    <div
      onClick={onClose}
      className={`flex items-center gap-3 px-5 py-3 bg-white dark:bg-chest-800/95 rounded-xl

```

```

shadow-floating border ${bgColors[toast.type]} animate-slide-up pointer-events-auto
cursor-pointer min-w-[280px] max-w-md`
    >
      {icons[toast.type]}
      <p      className="flex-1      text-base      font-medium      text-charcoal
dark:text-parchment">{toast.message}</p>
      <button
        onClick={(e) => { e.stopPropagation(); onClose(); }}
        className="p-1 hover:bg-parchment/20 dark:hover:bg-chest-700/40 rounded-lg
transition-colors cursor-pointer"
      >
        <X size={18} className="text-taupe" />
      </button>
    </div>
  );
}

```

```

export function useToast() {
  const context = useContext(ToastContext);
  if (!context) {
    return {
      showToast: (message: string, type?: ToastType) => {
        alert(message);
      },
      showAlert: (message: string, type?: ToastType) => {
        alert(message);
      },
    };
  }
  return context;
}

```

```

import { clsx, type ClassValue } from 'clsx';
import { twMerge } from 'tailwind-merge';

```

```

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

```

```
/**
```

```
 * 检查字符串是否为有效 URL
```

```
*/
```

```

export function isValidUrl(url: string): boolean {
  try {

```



```
        const urlObj = new URL(url);
        return urlObj.protocol === 'http:' || urlObj.protocol === 'https:';
    } catch {
        return false;
    }
}

/**
 * 解析分享文本，提取其中的 URL
 */
export function parseShareText(text: string): { isShareText: boolean; url?: string } {
    const urlRegex = /(https?:\/\/[^\s]+)/g;
    const matches = text.match(urlRegex);
    if (matches && matches.length > 0) {
        return { isShareText: true, url: matches[0] };
    }
    return { isShareText: false };
}

/**
 * 从 URL 解析平台标识
 */
export function parseUrlPlatform(url: string): string {
    try {
        const urlObj = new URL(url);
        const host = urlObj.hostname.toLowerCase();

        const platformMap: Record<string, string> = {
            'douyin.com': 'douyin',
            'iesdouyin.com': 'douyin',
            'xiaohongshu.com': 'xiaohongshu',
            'xhslink.com': 'xiaohongshu',
            'bilibili.com': 'bilibili',
            'b23.tv': 'bilibili',
            'weibo.com': 'weibo',
            'weibo.cn': 'weibo',

            'zhihu.com': 'zhihu',
            'taobao.com': 'taobao',
            'tmall.com': 'taobao',
            'jd.com': 'jd',
            'youtube.com': 'youtube',
            'youtu.be': 'youtube',
            'twitter.com': 'twitter',
        };
    }
}
```

```
        'x.com': 'twitter',
        'instagram.com': 'instagram',
        'tiktok.com': 'tiktok',
        'amazon.com': 'amazon',
        'github.com': 'github',
    };

    for (const [domain, platform] of Object.entries(platformMap)) {
        if (host.includes(domain)) {
            return platform;
        }
    }

    return 'other';
} catch {
    return 'other';
}
}

'use client';

import { useState, useRef, useCallback, useEffect } from 'react';
import { Link2, Sparkles, Palette, Loader2, ImageIcon, RefreshCw, Library, Lock } from
' lucide-react';
import { api, type ApiError } from '@lib/api';
import { getUser } from '@lib/auth';
import { use18n } from '@lib/i18n';
import { generateDefaultCover } from '@lib/platforms';
import { cn } from '@lib/utils';
import { useToast } from '@components/Toast';
import { useQuery } from '@tanstack/react-query';

type CoverMode = 'url' | 'ai' | 'gradient' | 'library';

interface CoverEditorProps {
    value: string;
    platform: string;
    onChange: (value: string) => void;
    disabled?: boolean;
    collectionId?: string;
}

/** 根据 coverImage 值推断用户上次选择的 mode */
function inferModeFromValue(val: string): CoverMode {
```

```
    if (!val || val.startsWith('data:image/svg')) return 'gradient';
    if (val.startsWith('data:image/') || val.includes('cos.') || val.includes('myqcloud.com')) return
    'library';
    return 'url';
  }
}
```

```
export default function CoverEditor({ value, platform, onChange, disabled, collectionId }:
CoverEditorProps) {
```

```
  const { t } = useI18n();
  const { showToast, showAlert } = useToast();
  const [mode, setMode] = useState<CoverMode>(inferModeFromValue(value));
  const [uploading, setUploading] = useState(false);
  const [syncing, setSyncing] = useState(false);
  const [previewUrl, setPreviewUrl] = useState(value || '');
  const [dragOver, setDragOver] = useState(false);
  const fileInputRef = useRef<HTMLInputElement>(null);
  const initialCheckDoneRef = useRef(false);
```

```
  const gradientSvg = generateDefaultCover(platform);
```

```
  // 系统封面库查询（必须在 useEffect 之前声明）
```

```
  const { data: systemCoversData, refetch: refetchSystemCovers } = useQuery({
    queryKey: ['systemCovers'],
    queryFn: async () => {
      const response = await api.get('/upload/system-covers');
      return response.data;
    },
    enabled: true,
  });
```

```
  // 仅在组件首次挂载且系统封面数据加载完成后，若当前值匹配 AI 封面则自动切换到
  ai 模式
```

```
  // 避免用户手动切换模式后被强制拉回
```

```
  useEffect(() => {
    if (!value || !initialCheckDoneRef.current || !systemCoversData?.data) return;
    const isSystemCover = systemCoversData.data.some((c: { cosUrl: string }) => c.cosUrl ===
value);
    initialCheckDoneRef.current = true;
    if (isSystemCover && mode !== 'ai') {
      setMode('ai');
    }
  }, [systemCoversData?.data, value]);
```

```
  // 获取用户订阅级别，判断是否有封面上传权限（heavy / super 用户可用）
```

```
const user = typeof window !== 'undefined' ? getUser() : null;
const userTier = (user?.userTier as string) || 'medium';
const canUploadCover = userTier === 'heavy' || userTier === 'super';

// 封面库查询
const { data: coverLibraryData, refetch: refetchCovers } = useQuery({
  queryKey: ['coverLibrary'],
  queryFn: async () => {
    const response = await api.get('/upload/covers?limit=50');
    return response.data;
  },
  enabled: mode === 'library',
});

const compressImage = (file: File, maxWidth = 1200, maxHeight = 1200, quality = 0.8):
Promise<string> => {
  return new Promise((resolve, reject) => {
    const img = new Image();
    img.onload = () => {
      const canvas = document.createElement('canvas');
      let { width, height } = img;
      if (width > maxWidth || height > maxHeight) {
        const ratio = Math.min(maxWidth / width, maxHeight / height);
        width = Math.floor(width * ratio);
        height = Math.floor(height * ratio);
      }
      canvas.width = width;
      canvas.height = height;
      const ctx = canvas.getContext('2d')!;
      ctx.drawImage(img, 0, 0, width, height);
      const dataUrl = canvas.toDataURL('image/jpeg', quality);
      URL.revokeObjectURL(img.src);
      resolve(dataUrl);
    };
    img.onerror = () => {
      URL.revokeObjectURL(img.src);
      reject(new Error('图片加载失败，可能是不支持的格式'));
    };
    img.src = URL.createObjectURL(file);
  });
};
```